TechPost ELEVATED

DC – AI – Routing – Switching – Security – Wireless – Automation

Using Junos SNMP utility MIB on Juniper SRX

An HPE Juniper Networking Techpost

Karel Hendrych

2024-02-07

Contents

1 Using Junos SNMP utility MIB on Juniper SRX 2
1.1 Using utility MIB from on-box Python 2
1.2 Useful links 4
1.3 Glossary 4

1 Using Junos SNMP utility MIB on Juniper SRX

Karel Hendrych - 02/07/2024

Although good old Junos SNMP MIB is very rich on every platform, occasionally some specific stats could have been handy. For example, number of sessions per IP protocol on SRX. No problem! Blast from the past Junos utility MIB tooling allows expansion of MIB by anything retrievable using RPCs. This short Tech Post aims to give a good starting point for daily use of this simple yet powerful approach.

1.1 Using utility MIB from on-box Python

In this specific example script in background will be automatically collecting SRX session counts for specific IP protocol followed by load to utility MIB. Related RPC summarizing number of connections in SRX firewall session table is represented on CLI by:

```
root@srx> show security flow session protocol tcp summary
...
Total sessions: 125
```

Following on-box Python skeleton code loads session counts for individual IP protocols defined by item values of *ip_proto_session_counts* dictionary, key is used for naming the SNMP counter:

```
ip_proto_session_counts = {
   "session_count_tcp": "tcp",
   "session_count_udp": "udp",
   "session_count_icmp": "icmp",
   "session_count_gre": "47",
}


from jnpr.junos import Device


def set_mib(instance, object_value):
   dev.rpc.request_snmp_utility_mib_set(
      object_type="counter64",
      instance=instance,
      object_value=object_value,
   )


with Device() as dev:
   for (
      snmp_counter_name,
      protocol,
   ) in ip_proto_session_counts.items():
      result = dev.rpc.get_flow_session_information(
         protocol=protocol, summary=True
      )
      session_count = result.findall(
         ".//displayed-session-count"
      )[0].text
      set_mib(snmp_counter_name, session_count)
```

Script is placed for manual execution (handy for testing changes) in */var/db/scripts/op* folder and for periodic execution also in */var/db/scripts/event*. Good practice may be simply to create a hard-link instead of making and maintaining a copy:

```
root@srx:/var/db/scripts/event # ln ../op/mib.py mib.py
```

Then the Junos configuration side for both manual and periodic execution (every 60s in sample below):

```
set system scripts op file mib.py command mib
set system scripts language python3
set system login user python-script-user class super-user
```

```
set event-options generate-event one_minute time-interval 60
set event-options policy mib events one_minute
set event-options policy mib then event-script mib.py
set event-options event-script file mib.py python-script-user python-script-user
```

Finally, to execute the script manually for a test-drive on CLI and retrieve counters from utility MIB:

```
root@srx> op mib
root@srx> show snmp mib walk jnxUtil ascii
jnxUtilCounter64Value."session_count_gre" = 2
jnxUtilCounter64Value."session_count_icmp" = 13
jnxUtilCounter64Value."session_count_tcp" = 129
jnxUtilCounter64Value."session_count_udp" = 106
```

For any expansions, to reveal RPC with parameters and corresponding XML data representation using Junos CLI (reduced output for specific example):

```
root@srx> show security flow session protocol tcp summary | display xml rpc
<rpc-reply>
    <rpc>
        <get-flow-session-information>
              <protocol>tcp</protocol>
              <summary/>
        </get-flow-session-information>
    </rpc>
    <cli>
        <banner></banner>
    </cli>
</rpc-reply>
```

Notice the "_" character usage in Python code instead of non-allowed "-" in RPC name above. A common pitfall is to copy paste the RPC as-is into Python code.

And related output XML data structure where from data are extracted:

```
root@srx> show security flow session protocol tcp summary | display xml
<rpc-reply>
    <security-flow-information>
      <flow-session-information>
          <displayed-session-valid>128</displayed-session-valid>
          <displayed-session-pending>0</displayed-session-pending>
          <displayed-session-other>0</displayed-session-other>
          <displayed-session-count>128</displayed-session-count>
       </flow-session-information>
    </security-flow-information>
    <cli>
        <banner></banner>
    </cli>
</rpc-reply>
```

Notes

- Of course, when doing similar things for a real-world system, then security, exception handling, logging and RE/PFE load imposed by executed code MUST be considered.
- Parameters can be passed to the script from Junos config and CLI. E.g., the ip_proto_session_counts data structure in some string form as parameter to have Junos config driven counter definition instead of script contained variable. Handy is Python split() method to create a list from string.
- To avoid nagging in Junos logs about execution of unsigned script, SHA-256 checksum of the script file needs to be part of Junos config:

```
root@vsrx:~ # sha256 /var/db/scripts/op/mib.py
...

set event-options event-script file mib.py checksum sha-256 ...
set system scripts op file mib.py checksum sha-256 ...
```

- Simple trick for clearing counters when starting over in development environment:

```
root@srx> restart snmp
```

- Generally, there may be better ways for close to real-time data collection use-cases, e.g., if there are appropriate telemetry sensors.

1.2 Useful links

- Junos Scripting: https://www.juniper.net/documentation/us/en/software/junos/automation-scripting/topics/concept/junos-script-automation-overview.html
- Utility MIB: https://www.juniper.net/documentation/en_US/junos/topics/task/operational/security-snmp-best-practices-utility-mib-using.html
- PyEZ developer guide: https://www.juniper.net/documentation/us/en/software/junos-pyez/junos-pyez-developer/index.html

1.3 Glossary

- CLI: Command Line Interface
- MIB: Management Information Base
- PFE: Packet Forwarding Engine
- RE: Routing Engine
- RPC: Remote Procedure Call
- SNMP: Simple Network Management Protocol



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.
Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.
Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com



